

PaceUp

Guía de instalación y arranque en local

App Flutter · Windows, macOS y Linux · Chrome, Android e iOS

Índice

1. Qué es y contra qué corre	2
2. Requisitos por sistema operativo	2
2.1. Instalar Flutter	2
2.2. Dejar flutter doctor en verde	3
3. Clonar la app y preparar dependencias	4
4. Opción A — Levantar la app contra el backend oficial	5
4.1. En Chrome	5
4.2. En Android	5
4.3. En iOS (solo macOS)	5
4.4. Entrar en la app	6
5. Por qué el puerto 5000 es obligatorio en web	7
5.1. Qué no funciona igual en Chrome	7
6. Opción B — Backend en tu máquina	8
6.1. Paso 1 — Instalar Node, PostgreSQL y Redis	8
6.2. Paso 2 — Crear el rol y las bases de datos	8
6.3. Paso 3 — Clonar y configurar la API	9
6.4. Paso 4 — Migrar, sembrar y arrancar	9
6.5. Paso 5 — Apuntar la app al backend local	10
6.6. Cómo se decide la URL base	10
7. Compilación	11
7.1. Android — APK y AAB	11
7.2. iOS — IPA (solo macOS)	11
7.3. Web	11
8. Referencia: make frente a comando directo	12
9. Qué esperar de la app	12
10. Problemas frecuentes	12
11. Documentación relacionada	13

1. Qué es y contra qué corre

PaceUp es la app móvil de running en Flutter: plan de entrenamiento, seguimiento de carreras en vivo con GPS, historial persistente e inscripción a maratones.

Hay **dos formas** de levantarla, y esta guía cubre las dos por separado:

Opción	Cuándo usarla	Sección
A — Backend oficial	Vas a tocar solo la app. No instalas nada de servidor. Es lo normal.	cap. 4
B — Backend en tu máquina	Vas a tocar también la API. Clonas runner-api, instalas Postgres y Redis.	cap. 6

El backend oficial ya está desplegado y funcionando: <https://cam-run.tumype.com/api/v1>. No hace falta montar nada de servidor para trabajar en la app. Empieza por la Opción A.

Auth, Home y el catálogo de maratones hablan ya con el backend real. Train, Races y Profile todavía usan repositorios en memoria (ver ARCHITECTURE.md, sección 9).

URL base	https://cam-run.tumype.com/api/v1
Panel admin	https://cam-run.tumype.com/admin
Moneda / zona	BOB · America/La_Paz
Autenticación	JWT. El login acepta correo o CI en el mismo campo
Cuenta con datos	runner@test.com / Test1234!

2. Requisitos por sistema operativo

	Windows	macOS	Linux
Flutter 3.44+ (Dart 3.12)	Sí	Sí	Sí
Chrome	Sí	Sí	Sí
Android Studio + SDK	Sí	Sí	Sí
Xcode + CocoaPods	No disponible	Solo para iOS	No disponible
make	No viene; opcional	Con Xcode CLT	Paquete del sistema
Compilar APK / AAB	Sí	Sí	Sí
Compilar IPA (iPhone)	Imposible	Sí	Imposible

iOS solo se compila en macOS. No es una limitación de este proyecto: Apple no distribuye Xcode para Windows ni Linux. Desde Windows o Linux puedes trabajar en todo el código de la app, pero para generar un .ipa o correr en un iPhone necesitas un Mac (o un runner macOS en CI).

2.1. Instalar Flutter

2.1.1. Windows

Con winget, en PowerShell:

```
winget install --id Git.Git -e
winget install --id Google.AndroidStudio -e
```

Flutter conviene instalarlo a mano, porque la ruta importa:

1. Descarga el ZIP de docs.flutter.dev.
2. Descomprímelo en C:/src/flutter. **Sin espacios ni acentos en la ruta**, y fuera de C:/Program Files (necesita permisos de escritura).
3. Añade C:/src/flutter/bin al PATH del usuario:

```
[Environment]::SetEnvironmentVariable(
    "Path", $env:Path + ";C:\src\flutter\bin", "User")
```

4. Cierra y abre PowerShell, y comprueba:

```
flutter --version
flutter doctor
```

2.1.2. macOS

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
brew install --cask flutter
brew install --cask android-studio
xcode-select --install          # trae git y make
flutter --version
flutter doctor
```

Para iOS, además de Xcode desde la App Store:

```
sudo xcodebuild -license accept
sudo xcode-select -s /Applications/Xcode.app/Contents/Developer
sudo gem install cocoapods
```

2.1.3. Linux (Ubuntu / Debian)

```
sudo apt update
sudo apt install -y git curl unzip xz-utils zip libglu1-mesa make
sudo snap install flutter --classic      # o el tar.xz de docs.flutter.dev
sudo snap install android-studio --classic
flutter --version
flutter doctor
```

En Fedora o Arch, cambia apt por dnf o pacman y descarga Flutter desde docs.flutter.dev descomprimiendo en ~/development/flutter, con export PATH="\$PATH:\$HOME/development/flutter/bin" en tu ~/.bashrc o ~/.zshrc.

2.2. Dejar flutter doctor en verde

En los tres sistemas, lo que casi siempre falta:

```
flutter doctor --android-licenses      # acepta todas con "y"
```

Y en Android Studio, **Settings** → **Languages & Frameworks** → **Android SDK**: instala el **Android SDK Command-line Tools**, sin el cual flutter doctor no valida las licencias.

No hace falta que **todo** esté en verde. Para trabajar en la app basta con:

Chrome	Chrome - develop for the web en verde
Android	Android toolchain y Android Studio en verde
iOS	Xcode en verde (solo macOS)

3. Clonar la app y preparar dependencias

Igual en los tres sistemas:

```
git clone https://github.com/alvaro-tc/running-app.git
cd running-app
flutter pub get
flutter devices
```

`flutter devices` es la comprobación que ahorra media hora de dudas: si el dispositivo en el que quieres correr no aparece ahí, tampoco lo va a encontrar `flutter run`.

4. Opción A — Levantar la app contra el backend oficial

No instalas nada de servidor. La URL ya está en el archivo `.env` de la raíz del repo y entra en el binario como constante de compilación.

4.1. En Chrome

El puerto debe ser **5000**. Es obligatorio, y el porqué está en el capítulo 5.

Windows PowerShell	<code>flutter run -d chrome --web-port=5000 --dart-define-from-file=.env</code>
macOS Terminal	<code>make run-web</code>
Linux bash	<code>make run-web</code>

`make run-web` es exactamente ese mismo comando de Flutter; el Makefile solo lo guarda. En Windows no hay `make`, por eso se escribe entero.

4.2. En Android

Funciona igual en Windows, macOS y Linux. Primero necesitas un dispositivo.

4.2.1. Con un emulador

```
flutter emulators          # lista los que ya existen
flutter emulators --launch <id>  # arranca uno
```

Si no hay ninguno, créalo desde Android Studio en **Device Manager** → **Create Device**: elige un Pixel con imagen de sistema **Google Play** (las imágenes «Google APIs» sin Play también sirven, pero las de Play traen los servicios de localización completos).

4.2.2. Con un teléfono físico

1. En el teléfono: **Ajustes** → **Acerca del teléfono**, toca siete veces **Número de compilación** para activar las opciones de desarrollador.
2. **Ajustes** → **Opciones de desarrollador**: activa **Depuración por USB**.
3. Conecta por USB y acepta el diálogo de confianza que sale en la pantalla.
4. Solo en Linux, si el teléfono no aparece, faltan las reglas udev: `sudo apt install android-sdk-platform-tools-common`.

4.2.3. El comando

```
flutter devices          # que aparezca tu Android
flutter run --dart-define-from-file=.env  # los tres sistemas
```

o `make run` en macOS y Linux. Si hay más de un dispositivo conectado, elige con `-d`:

```
flutter run -d emulator-5554 --dart-define-from-file=.env
```

Mientras corre: `r` recarga en caliente, `R` reinicia, `q` sale.

4.3. En iOS (solo macOS)

4.3.1. En el simulador

```
open -a Simulator
flutter devices
flutter run -d "iPhone 16" --dart-define-from-file=.env
```

4.3.2. En un iPhone físico

1. Conecta el iPhone y confía en el Mac desde el diálogo del teléfono.
2. Abre `ios/Runner.xcworkspace` en Xcode → pestaña **Signing & Capabilities** → elige tu **Team**. Una cuenta de Apple gratuita vale para desarrollo; el perfil caduca a los 7 días.

3. En el iPhone: **Ajustes** → **General** → **VPN y gestión de dispositivos**, y confía en tu certificado de desarrollador.

```
cd ios && pod install && cd ..  
flutter run -d <id-del-iphone> --dart-define-from-file=.env
```

El GPS de verdad solo se prueba en un teléfono. El simulador de iOS y el emulador de Android inventan posiciones, y el navegador para el GPS al cambiar de pestaña. En modo debug la app usa además `SimulatedLocationService`, que reproduce una ruta pregrabada 20 veces más rápido; para GPS real en un dispositivo, sobrescribe `useSimulatedLocationProvider` con `false`.

4.4. Entrar en la app

Al arrancar por primera vez verás el onboarding. Entra con:

Correo	Contraseña	Qué tiene
runner@test.com	Test1234!	La cuenta con datos: 4 meses de entrenamientos, plan activo, inscripciones y zapatillas
admin@test.com	Test1234!	Panel de administración en <code>/admin</code>

El campo del login acepta también la CI (6789012LP). El resto de cuentas están en `docs/cuentas-de-prueba.md`.

Si el catálogo sale vacío o el login da 401, la base de producción está sin sembrar. Se arregla en el VPS con `cd /srv/running-api && npm run db:seed`. Mientras tanto puedes crear una cuenta desde la pantalla de registro de la propia app.

5. Por qué el puerto 5000 es obligatorio en web

El navegador exige que el origen desde el que se hacen las peticiones esté autorizado por el servidor mediante CORS, y el backend solo tiene en su lista blanca `http://localhost:5000`. Si dejas que `flutter run -d chrome` elija puerto, elegirá uno **distinto cada vez**, y un puerto aleatorio no se puede meter en una lista blanca.

Síntoma cuando falta: la app se queda en el esqueleto de carga. El servidor responde 200 y el navegador tira la respuesta sin que la app se entere.

Comprobación —debe devolver la cabecera:

```
curl -sI -H "Origin: http://localhost:5000" \
https://cam-run.tumype.com/api/v1/config/app | grep -i access-control-allow-origin
```

En Windows, con PowerShell:

```
(Invoke-WebRequest -Uri "https://cam-run.tumype.com/api/v1/config/app" `
-Headers @{Origin="http://localhost:5000"}).Headers
```

Del lado del servidor, en `/etc/running-api/.env.production`:

```
CORS_ORIGINS=https://cam-run.tumype.com,http://localhost:5000
```

```
sudo systemctl restart running-api
```

Si el puerto 5000 está ocupado en tu máquina, libéralo. Cambiarlo en el comando rompe todas las peticiones hasta que alguien añada el nuevo origen en el VPS.

5.1. Qué no funciona igual en Chrome

Grabar en segundo plano	No. El GPS del navegador se para al cambiar de pestaña
Tokens	Van a <code>localStorage</code> , no al <code>Keychain</code> : vale para mirar pantallas, no para juzgar seguridad
Mapas y resto de la UI	Igual que en el teléfono

Web sirve para ver pantallas rápido. Lo que se prueba de verdad —tracking, permisos, segundo plano— se prueba en un teléfono.

6. Opción B — Backend en tu máquina

Solo si vas a tocar la API. El repositorio es github.com/alvaro-tc/runner-api.

No se usa Docker en ningún entorno. Postgres y Redis se instalan nativos.

Node.js	20 o superior (probado en 22 LTS)
PostgreSQL	16 o superior (probado en 18.4)
Redis	7

6.1. Paso 1 — Instalar Node, PostgreSQL y Redis

6.1.1. Windows

```
winget install --id OpenJS.NodeJS.LTS -e
winget install --id PostgreSQL.PostgreSQL.16 -e
```

Redis **no tiene build oficial para Windows**. Dos salidas, en orden de menos fricción:

Opción 1 — WSL2 (recomendado):

```
wsl --install -d Ubuntu
```

y dentro de Ubuntu:

```
sudo apt update && sudo apt install -y redis-server
sudo service redis-server start
redis-cli ping          # PONG
```

Desde Windows se ve como `redis://localhost:6379`: WSL2 reenvía el puerto solo.

Opción 2 — Memurai, que es Redis portado a Windows:

```
winget install --id Memurai.MemuraiDeveloper -e
```

Comprueba las versiones (reabre PowerShell antes):

```
node --version
psql --version
```

Si psql no se reconoce, añade `C:/Program Files/PostgreSQL/16/bin` al PATH.

6.1.2. macOS

```
brew install node@22 postgresql@16 redis
brew services start postgresql@16
brew services start redis
node --version && psql --version && redis-cli ping
```

6.1.3. Linux (Ubuntu / Debian)

```
curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -
sudo apt install -y nodejs postgresql redis-server
sudo systemctl enable --now postgresql redis-server
node --version && psql --version && redis-cli ping
```

6.2. Paso 2 — Crear el rol y las bases de datos

Abre una consola de Postgres como superusuario:

Windows	<code>psql -U postgres</code> (pide la contraseña del instalador)
----------------	---

macOS	psql postgres
Linux	sudo -u postgres psql

y dentro, en los tres casos:

```
CREATE ROLE paceup LOGIN PASSWORD 'paceup';
CREATE DATABASE paceup OWNER paceup; -- desarrollo
CREATE DATABASE paceup_shadow OWNER paceup; -- solo prisma migrate dev
CREATE DATABASE paceup_test OWNER paceup; -- tests e2e
```

Sal con \q. paceup_shadow la usa prisma migrate dev para detectar drift: se puede borrar y recrear sin perder nada.

6.3. Paso 3 — Clonar y configurar la API

```
git clone https://github.com/alvaro-tc/runner-api.git
cd runner-api
npm install
```

Copia la plantilla de entorno:

```
cp .env.example .env # macOS / Linux
```

```
Copy-Item .env.example .env # Windows
```

Y edita .env. Lo mínimo que hay que tocar:

DATABASE_URL	postgresql://paceup:paceup@localhost:5432/paceup?schema=public
REDIS_URL	redis://localhost:6379
JWT_SECRET	Sin default a propósito. Genera el tuyo (abajo)
CORS_ORIGINS	* en desarrollo, o http://localhost:5000 para ser estricto

```
node -e "console.log(require('crypto').randomBytes(48).toString('base64'))"
```

Si falta una variable o tiene un valor imposible, **el proceso muere al arrancar** diciendo exactamente cuál. Es a propósito: nunca se levanta a medias.

6.4. Paso 4 — Migrar, sembrar y arrancar

```
npm run db:migrate # crea el esquema
npm run db:generate # genera el cliente de Prisma
npm run db:seed # cuentas, catálogo y un corredor con historial
npm run dev # escucha en :3000
```

Sin db:seed no hay maratones ni cuentas de prueba: la app arranca, llega al login, y no hay con qué entrar.

Verifica antes de tocar la app:

```
curl -s http://localhost:3000/health # 200: el proceso responde
curl -s http://localhost:3000/ready # 200 si Postgres y Redis responden
```

```
curl -s http://localhost:3000/api/v1/config/app # constantes del entorno
curl -s "http://localhost:3000/api/v1/marathons?limit=3" # catalogo publico
```

La documentación interactiva queda en <http://localhost:3000/api/docs> y el panel de administración en <http://localhost:3000/admin>.

6.5. Paso 5 — Apuntar la app al backend local

Deja `npm run dev` corriendo y abre **otra terminal** en `running-app`.

Cuál es la URL depende de dónde corre la app, no de tu sistema operativo:

Dónde corre la app	Host	Comando
Emulador de Android	10.0.2.2	<code>flutter run --dart-define-from-file=.env.local</code>
Simulador de iOS	localhost	<code>flutter run --dart-define=API_BASE_URL=http://localhost:3000/api/v1</code>
Chrome	localhost	<code>flutter run -d chrome --web-port=5000 --dart-define=API_BASE_URL=http://localhost:3000/api/v1</code>
Teléfono físico	IP del equipo	<code>flutter run --dart-define=API_BASE_URL=http://192.168.1.50:3000/api/v1</code>

En macOS y Linux, el primero es `make run-local`.

10.0.2.2 solo existe dentro del emulador de Android: es el alias con el que ve al equipo anfitrión. Desde un teléfono real no resuelve a nada. Averigua tu IP con `ipconfig` (Windows) o `ip addr / ifconfig` (macOS y Linux), y asegúrate de que el teléfono está en la misma red Wi-Fi. Si aun así no conecta, suele ser el firewall del equipo bloqueando el 3000.

6.6. Cómo se decide la URL base

La URL base **no se escribe en el código**: entra como constante de compilación con `--dart-define-from-file`, que es de Flutter y no necesita ningún paquete.

Archivo	Comando	URL base
<code>.env</code>	<code>make run</code>	<code>https://cam-run.tumype.com/api/v1</code>
<code>.env.local</code>	<code>make run-local</code>	<code>http://10.0.2.2:3000/api/v1</code>

Lo que hay en esos archivos se **incrusta en el binario**: ahí no va nunca un secreto, cualquiera con el APK puede leerlo. Sin ninguno de los dos, la app cae al backend local por defecto (`api_config.dart`).

7. Compilación

En los tres sistemas, `--dart-define-from-file=.env` **no es opcional**: sin él la URL que queda incrustada es 10.0.2.2, que solo existe para el emulador, y en un teléfono real no conecta con nada.

7.1. Android — APK y AAB

Funciona igual en Windows, macOS y Linux.

```
flutter build apk --release --dart-define-from-file=.env
# -> build/app/outputs/flutter-apk/app-release.apk
```

Para subir a Google Play hace falta el **App Bundle**, no el APK:

```
flutter build appbundle --release --dart-define-from-file=.env
# -> build/app/outputs/bundle/release/app-release.aab
```

APK más pequeño, uno por arquitectura:

```
flutter build apk --release --split-per-abi --dart-define-from-file=.env
```

Instalar el APK en un teléfono conectado:

```
flutter install
# o: adb install -r build/app/outputs/flutter-apk/app-release.apk
```

Un build de release **sin firmar con tu propia clave** usa la clave de debug: vale para probar, no para publicar. Para publicar hay que generar un keystore con `keytool -genkey -v -keystore upload-keystore.jks -keyalg RSA -keysize 2048 -validity 10000 -alias upload` y referenciarlo desde `android/key.properties`. Ese archivo **no se versiona**.

7.2. iOS — IPA (solo macOS)

```
cd ios && pod install && cd ..
flutter build ipa --release --dart-define-from-file=.env
# -> build/ios/ipa/*.ipa
```

Requiere una cuenta de Apple Developer de pago para distribuir, y firmar el archivo desde Xcode (**Product** → **Archive**) o con `--export-options-plist`. Desde Windows o Linux este comando no existe.

7.3. Web

```
flutter build web --release --dart-define-from-file=.env
# -> build/web/
```

Es HTML y JS estáticos: se sirven desde cualquier servidor. Para probar el build en local sin desplegarlo, **en el puerto 5000**:

```
cd build/web && python -m http.server 5000
```

El origen desde el que se sirva en producción tiene que estar en `CORS_ORIGINS` del backend, igual que `localhost:5000` en desarrollo.

8. Referencia: make frente a comando directo

En **Linux** y **macOS** make viene de serie y los targets son los comandos normales. En **Windows**, PowerShell no trae make, así que se llama a flutter directamente. La columna derecha funciona en los tres sistemas.

Target (Linux / macOS)	Comando directo (los tres sistemas)
make run	flutter run --dart-define-from-file=.env
make run-web	flutter run -d chrome --web-port=5000 --dart-define-from-file=.env
make run-local	flutter run --dart-define-from-file=.env.local
make apk	flutter build apk --release --dart-define-from-file=.env
make fmt	dart format .
make analyze	flutter analyze
make test	flutter test
make goldens	flutter test --update-goldens

En Windows, si prefieres usar el Makefile tal cual, MSYS2 trae mingw32-make en C:/msys64/mingw64/bin: añade esa carpeta al PATH y llama mingw32-make run-web.

En macOS make llega con las Command Line Tools de Xcode (xcode-select --install si faltan). En Linux, sudo apt install make.

9. Qué esperar de la app

Home	Cuenta atrás del próximo maratón, tarjeta del evento, plan semanal con anillos de progreso y sesión del día
Train	Inicio rápido, resumen semanal, historial agrupado y filtrable
Races	Totales calculados, inscripciones próximas y completadas con resultados
Profile	Estadísticas, calzado, sueño, hidratación y ajustes

Flujos completos disponibles: onboarding y auth (sign in / sign up / recuperar contraseña); detalle de maratón e inscripción en 3 pasos con dorsal; sesión de running a pantalla completa con mapa, splits y auto-pausa; resumen post-entrenamiento persistido; perfil editable. Tema claro y oscuro conmutables desde **Profile** → **Appearance**.

En modo debug, /dev/showcase muestra el catálogo de componentes del design system con un interruptor de tema.

10. Problemas frecuentes

make no se reconoce (Windows)	Normal: PowerShell no lo trae. Usa la columna derecha de la tabla de referencia
Pantalla de carga infinita en Chrome	CORS. Confirma que arrancaste con --web-port=5000 y que el origen está en CORS_ORIGINS
401 INVALID_CREDENTIALS	La base no está sembrada (npm run db:seed), o la contraseña no es Test1234!
Catálogo de maratones vacío	Falta npm run db:seed en el backend
El teléfono físico no conecta	Estás usando 10.0.2.2, que solo existe en el emulador. Pasa la IP del equipo por --dart-define

La API muere al arrancar	Falta una variable en <code>.env</code> . El mensaje dice cuál. Suele ser <code>JWT_SECRET</code>
/ready devuelve 503	Postgres o Redis no están levantados
flutter doctor se queja de licencias	<code>flutter doctor --android-licenses</code> , y acepta todas con «y»
Errores raros tras cambiar de rama	<code>flutter clean</code> && <code>flutter pub get</code>
Cambió la URL del backend y sigue usando la vieja	<code>flutter clean</code> : el binario cacheado tiene la constante vieja incrustada

11. Documentación relacionada

- `README.md` — resumen del proyecto y puesta en marcha.
- `docs/cuentas-de-prueba.md` — cuentas sembradas, entornos y estado de producción.
- `ARCHITECTURE.md` — capas, convenciones y cómo añadir una feature.
- `runner-api/README.md` — puesta en marcha detallada del backend.
- `runner-api/docs/pago-qr-manual.md` — cobro por QR con verificación manual.